01 Project Overview & System Architecture

> The Insurance Document Classifier is a browser-based ML application that classifies insurance documents (PDF, Word, Excel, scanned images) into a two-level hierarchy — Category → Subcategory — using a user-trained ML model. No external APIs, no cloud, no data leaves the machine.

Capabilities

Capability	Detail
Ingest	PDF, DOCX, XLSX, PNG/JPG/TIFF/BMP/GIF
Extract	Native parsing for office formats; Tesseract OCR for scanned images
Label	2-level hierarchy: Category → Subcategory (e.g. Claims → Medical Records)
Train	4 scikit-learn algorithms, adaptive cross-validation
Predict	Top-5 ranked results with probability confidence scores
Persist	Training data → JSON · model → joblib (local disk only)

Technology Stack

Layer	Library / Tool
UI	Streamlit ≥ 1.35
ML	scikit-learn ≥ 1.4 (pipelines, TF-IDF, cross-validation)
PDF	pdfplumber ≥ 0.10
Office docs	python-docx ≥ 1.1 · openpyxl ≥ 3.1 · pandas ≥ 2.0
OCR	Pillow ≥ 10 · pytesseract ≥ 0.3 + Tesseract system binary
Visualization	matplotlib ≥ 3.8 · seaborn ≥ 0.13
Serialization	joblib ≥ 1.3

Core Workflow

```
1. Upload labeled documents   →  training_data.json
2. Select ML algorithm        →  train the model once
3. Upload an unknown document →  get top-5 classification results
```

Architecture at a Glance

The system is a **single-process Streamlit application**. There is no separate backend API, no database server, and no message queue. All components run in-process; state is held in Streamlit's session state (in-memory) and on disk (JSON + joblib).

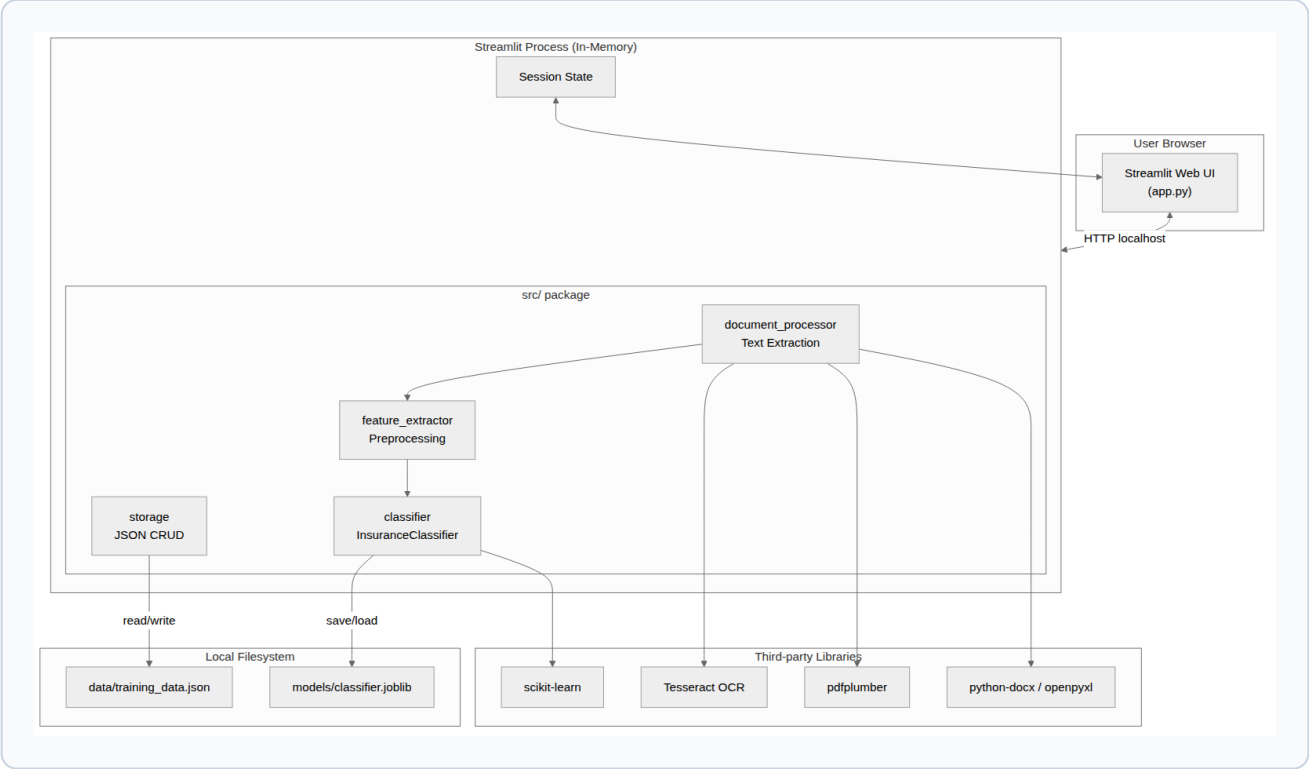
Component	Role
`app.py`	UI orchestration and page routing
`src/document_processor`	Extracts plain text from any supported format
`src/feature_extractor`	Normalizes text + applies keyword boosting
`src/classifier`	Wraps scikit-learn pipelines; train / predict / save / load
`src/storage`	CRUD over `data/training_data.json`
`src/defaults`	Built-in insurance taxonomy (6 categories)

Deployment

```
pip install -r requirements.txt
# Optional OCR support:
#   macOS:  brew install tesseract
#   Ubuntu: sudo apt-get install tesseract-ocr
streamlit run app.py  # opens http://localhost:8501
```

DIAGRAM 01

System Architecture



The Insurance Document Classifier is a monolithic single-process Streamlit app. The browser communicates via HTTP to the local Streamlit process, which hosts all business logic in the `src/` package. No external services or network calls are required — all persistence is handled through the local filesystem (JSON for training data, joblib for the serialized model).

02 Module Breakdown & Document Processing

src/ Package Modules

Module	Responsibility
`app.py`	UI orchestration — Train, Predict, Model Info pages
`document_processor`	Format-agnostic text extraction from any supported file type
`feature_extractor`	Text normalization + keyword boosting (×3 TF weight)
`classifier`	InsuranceClassifier — wraps pipelines, train/predict/save/load
`storage`	CRUD over `data/training_data.json`
`defaults`	Built-in taxonomy: 6 categories, 6–8 subcategories each

Default Insurance Taxonomy

Category	Example Subcategories
Policy	Auto, Home/Property, Health, Life, Commercial, Liability
Claims	Claim Form, Medical Records, Adjuster Report, Proof of Loss
Underwriting	Application Form, Risk Assessment, Loss History, Questionnaire
Financial	Premium Invoice, Payment Receipt, Loss Run Report, Audit Report
Legal	Endorsement, Exclusion Notice, Subrogation Letter, Power of Attorney
Correspondence	Cancellation Notice, Renewal Notice, Demand Letter, Declination Letter

User-defined categories merge with defaults at runtime via `get_all_categories()` — no config file needed.

Document Processing — Format Details

Format	Library	Extraction method
.pdf	pdfplumber	Page-by-page text extraction, joined with newlines
.docx / .doc	python-docx	Paragraphs + table cells (`" | "` separator)
.xlsx / .xls	openpyxl / pandas	All sheets linearized to `col1 | col2 / val1 | val2` rows
image formats	Pillow + Tesseract	OCR — requires Tesseract system binary

Feature Engineering

Keyword Boosting: User-provided keywords are appended **3×** to the feature string, inflating their TF-IDF weight without needing a custom vectorizer — a lightweight human-in-the-loop signal.

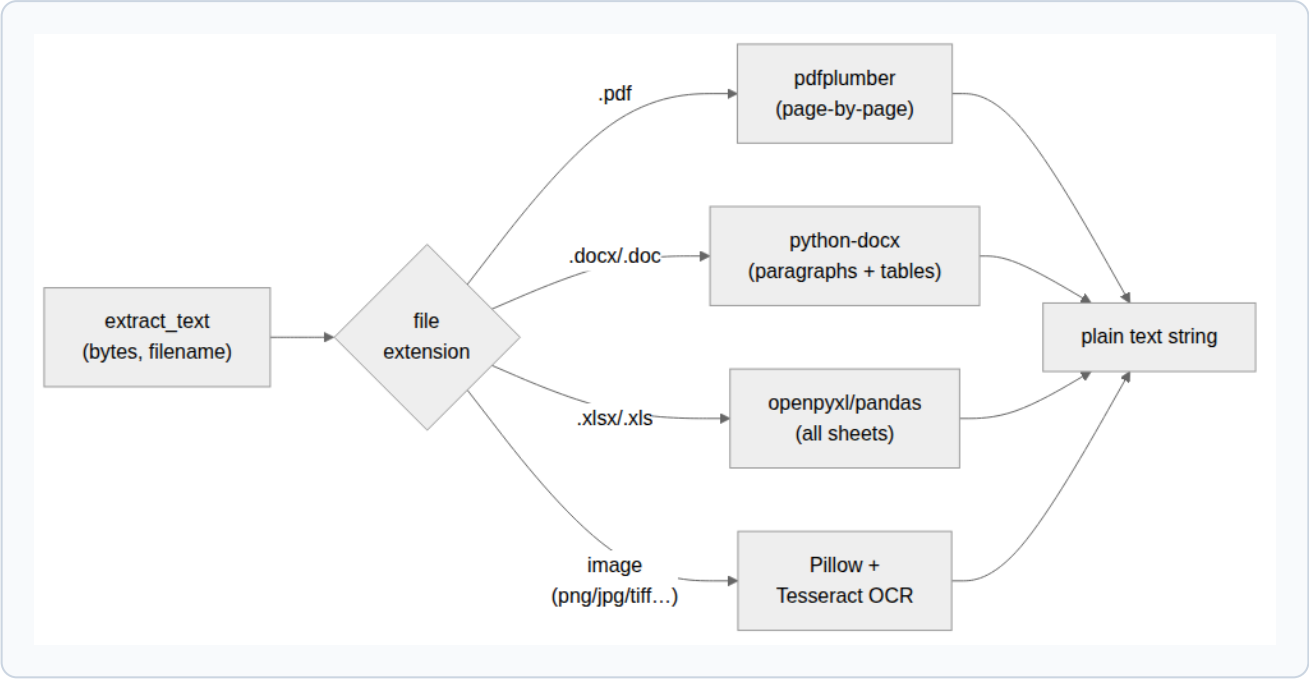
```
processed = lowercase(text)
processed = strip_non_alphanumeric(processed)
processed = collapse_whitespace(processed)
kw_boost  = " ".join(keywords) * 3
feature   = processed + " " + kw_boost
```

Directory Structure

```
insurance-doc-classifier/
├── app.py               # Streamlit entry point
├── requirements.txt
├── src/
│   ├── classifier.py    # ML engine
│   ├── defaults.py      # Insurance taxonomy
│   ├── document_processor.py
│   ├── feature_extractor.py
│   └── storage.py
├── data/                # Auto-created on first write
│   └── training_data.json
└── models/              # Auto-created on first train
    └── classifier.joblib
```

DIAGRAM 02

Document Processing Pipeline



`extract_text(bytes, filename)` is the single public entry point. It dispatches to a format-specific extractor based on the file extension and always returns a plain text string. This abstraction means the rest of the system (feature extraction, storage, classifier) never needs to know what file format was uploaded.

03 Training & Prediction Data Flows

Training Flow — Step by Step

#	Step	Function called
1	User uploads labeled document	`extract_text(bytes, filename)`
2	Assign category, subcategory, keywords	UI selectbox / text input
3	Persist to training store	`add_training_sample()` → JSON
4	Repeat for all training documents	—
5	Preprocess all stored samples	`prepare_training_texts(samples)`
6	Fit scikit-learn pipeline	`InsuranceClassifier.train(texts, labels)`
7	Adaptive cross-validation	`cross_val_score(cv=min(5, min_class_count))`
8	Serialize model to disk	`classifier.save("models/classifier.joblib")`

Cross-validation strategy: folds = `min(5, min_class_count)`. If any class has fewer than 2 samples, CV is skipped and training accuracy is used instead.

Prediction Flow — Step by Step

#	Step	Function called
1	Auto-load model on app startup	`InsuranceClassifier.load(MODEL_PATH)`
2	User uploads unknown document	`extract_text(bytes, filename)`
3	Preprocess + apply keyword boost	`prepare_single_text(text, keywords)`
4	Get ranked top-5 predictions	`classifier.predict_top_n(text, n=5)`
5	Parse compound label	`_parse_result()` — splits `"A|B"` on `"|"`
6	Display with confidence indicator	Green ≥70% · Orange ≥40% · Red <40%

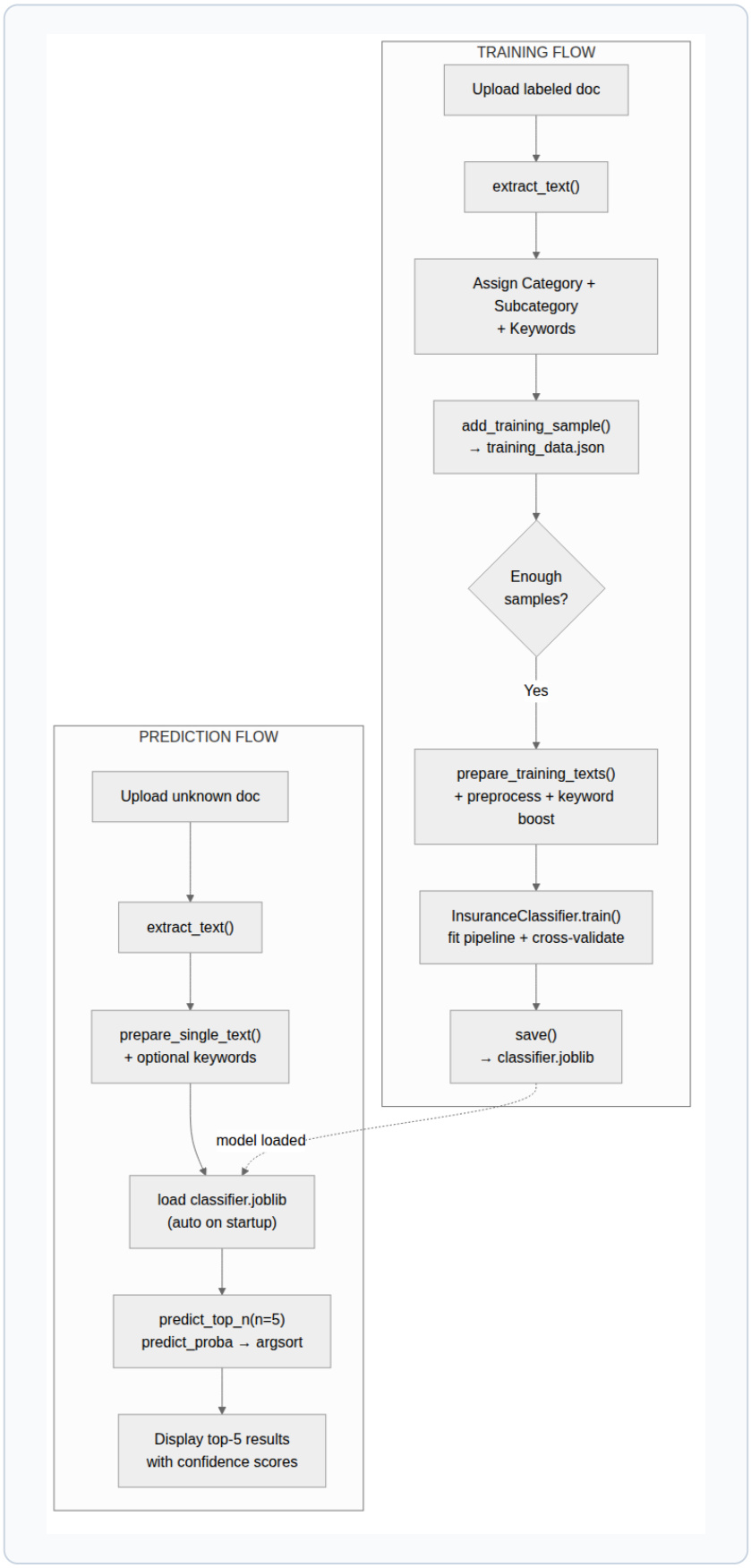
Compound label encoding: `"Claims|Medical Records"` is the flat string the model learns. After prediction, `_parse_result()` splits on `"|"` to recover both hierarchy levels without any model changes.

Prediction Output Schema

```
{
  "category":       "Claims",
  "subcategory":    "Medical Records",
  "compound_label": "Claims|Medical Records",
  "confidence":     0.87   # None if no
predict_proba
}
```

DIAGRAM 03

Training & Prediction Data Flow



The top subgraph shows the **Training Flow**: documents are uploaded and labeled one by one, persisted to JSON, then a single `InsuranceClassifier.train()` call fits the full pipeline and serializes the result to disk.

04 ML Pipeline Architecture & Algorithms

Two-Stage Pipeline Design

Every algorithm uses a scikit-learn `Pipeline` composed of exactly two steps: a **vectorizer** that converts the text feature string into a sparse numeric matrix, and an **estimator** that learns to predict compound labels from that matrix. The pipeline is serialized as a single object in `classifier.joblib`.

Algorithm Comparison

Algorithm	Vectorizer	Max features	predict_proba	Best for
Logistic Regression	TF-IDF	10,000	Native	General baseline, fast
Random Forest	TF-IDF	5,000	Native	Larger datasets, robust
SVM Linear	TF-IDF	10,000	Platt scaling[1]	High-dimensional text
Naive Bayes	CountVectorizer[2]	10,000	Native	Very small datasets

[1] `LinearSVC` wrapped in `CalibratedClassifierCV(cv=3)` to expose probability scores via Platt scaling.
[2] `MultinomialNB` requires non-negative integer input — raw term counts, not TF-IDF.

Vectorizer Settings (all algorithms)

Setting	Value	Purpose
ngram_range	(1, 2)	Capture single words and two-word phrases
sublinear_tf	True (TF-IDF only)	Log-scale term frequency to reduce outlier weight
max_features	5k–10k	Cap vocabulary size to control memory and speed

Model Serialization Schema

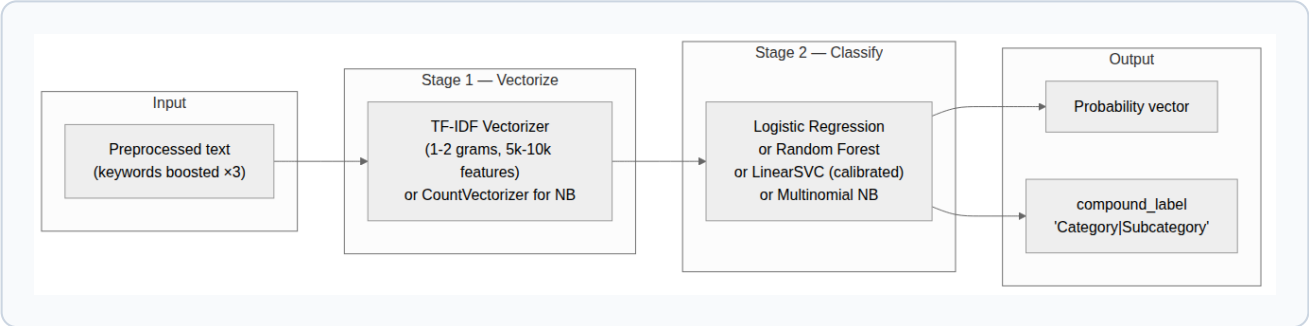
```
payload = {
  "pipeline":          Pipeline,     # fitted
sklearn Pipeline
  "classifier_name":   str,          # e.g.
"Logistic Regression"
  "training_accuracy": float,        # mean CV or
training acc
  "cv_scores":         ndarray|None, # per-fold CV
scores
  "classes_":          list[str],    # all compound
labels seen
  "n_samples_trained": int
}
```

Cross-Validation Logic

```
min_count = min(samples per class)
cv_folds  = min(5, min_count)  if min_count >= 2
else None

if cv_folds:
    cv_scores = cross_val_score(pipeline, texts,
                                labels,
cv=cv_folds)
    accuracy  = cv_scores.mean()
else:
    # fall back: measure on training data directly
    accuracy  = correct_predictions / total_samples
```

DIAGRAM 04

ML Pipeline Architecture



Each algorithm is implemented as a two-step scikit-learn `Pipeline`: **Stage 1** vectorizes the preprocessed text into a sparse numeric matrix (TF-IDF for LR/RF/SVM, raw counts for Naive Bayes); **Stage 2** classifies that matrix into a compound label (`"Category|Subcategory"`). All four pipelines expose `predict_proba` — SVM achieves this via Platt scaling calibration.

05 Data Model · UI Structure · Extension Points

Training Sample (JSON Schema)

```
{
  "id":             "uuid-v4",
  "filename":       "claim_form.pdf",
  "text":           "extracted full text…",
  "category":       "Claims",
  "subcategory":    "Medical Records",
  "keywords":       ["medical", "claim number"],
  "compound_label": "Claims|Medical Records",
  "timestamp":      "2026-06-22T14:23:01.456789"
}
```

Stored as a flat JSON array in `data/training_data.json`. Only `storage.py` reads or writes this file — all other modules go through its public API.

Three-Page UI

Page	Key Features
Train	Upload → assign category/subcategory + keywords → add to dataset → select algorithm → train → accuracy & CV scores
Test/ Predict	Upload → optional keywords → classify → top-5 ranked results → confidence progress bar (green/ orange/red)
Model Info	Metrics dashboard → class distribution chart → confusion matrix heatmap → training table → CSV export

Session State Keys

Key	Type	Purpose
`classifier`	`InsuranceClassifier|None`	Active trained model (auto-loaded from disk on startup)
`train_text`	`str|None`	Extracted text from the current training upload
`test_text`	`str|None`	Extracted text waiting for prediction

Error Handling

Scenario	Handling
Unsupported file type	`ValueError` shown as `st.error`
Tesseract not installed	`RuntimeError` with OS-specific install instructions
Corrupt PDF / DOCX	Exception caught in UI, shown as `st.error`
Fewer than 2 samples	Train button disabled; warning shown
Single-class dataset	Warning; training requires ≥ 2 distinct compound labels
Corrupt model file on disk	try/except → returns `None`, sidebar shows warning

Extension Points

Extension	Where to change
Add an ML algorithm	`classifier.py` — one new `elif` in `_build_pipeline()` + add to `ALGORITHM_NAMES`
Add a file format	`document_processor.py` — add to `SUPPORTED_EXTENSIONS` + new extractor function
Switch to a database	`storage.py` only — its public API stays the same
Expose a REST API	Wrap `classifier.predict` + `extract_text` in FastAPI
Swap in an LLM classifier	Implement the same `train / predict / save / load` interface

06 Bulk Training in Batches — Saving Time & Cost

The UI processes one document at a time — practical for small datasets, but slow for hundreds of pre-labeled files. Because the `src/` modules are fully importable as a Python library, a batch script can ingest an entire labeled archive and trigger **one single training run**, eliminating all redundant intermediate pipeline fits and cross-validation rounds.

Why Batching Saves Time & Cost

Approach	Training runs	100-doc effort
UI — one by one	Up to 100	~30–60 min, fully manual
Batch script	**1**	**~10–30 sec, fully automated**

Every `InsuranceClassifier.train()` call re-fits the full pipeline from scratch. Batching all samples first and training once avoids repeating the vectorizer fit and cross-validation for every intermediate dataset state.

Input Options

Input source	When to use
Labeled folder tree `/docs/Claims/Medical/*.pdf`	Files already organized into category subdirectories
CSV manifest `filename, category, subcategory, keywords`	Labels exist in a spreadsheet; files are in a flat folder
Existing JSON data	Migrating or merging data from another instance

Best practices:

- Aim for **≥ 10 samples per class** before training.
- Check mean CV accuracy — below 70% signals more or better-labeled data is needed.
- Use `clear_all_training_data()` before ingestion only when starting fresh; omit it to append to an existing dataset.

Batch Ingestion Script (CSV approach)

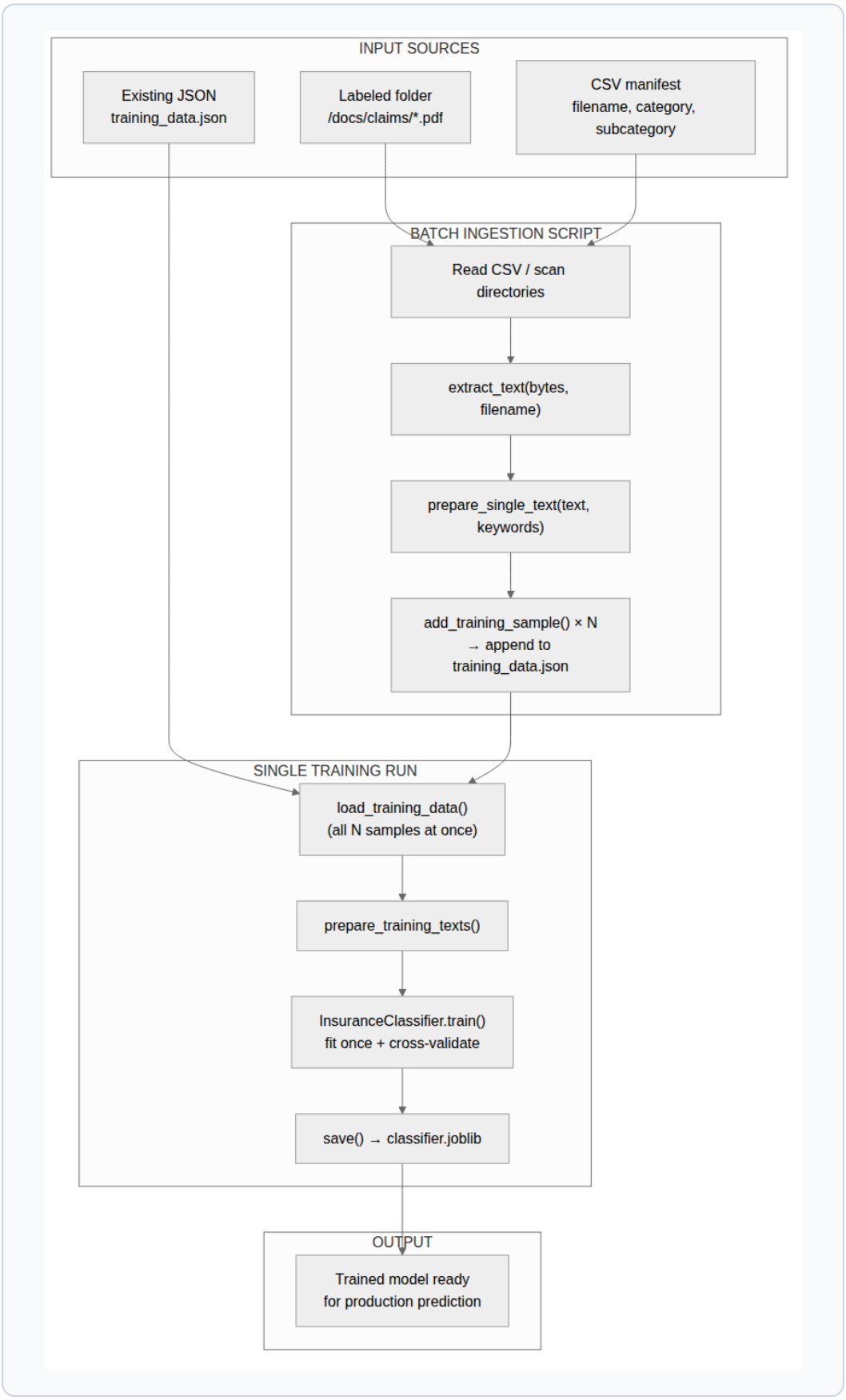
```
import csv, pathlib, sys
sys.path.insert(0, ".")          # make src/
importable
from src.document_processor import extract_text
from src.storage import (
    add_training_sample,
    clear_all_training_data,
    load_training_data,
)
from src.classifier import InsuranceClassifier
from src.feature_extractor import
prepare_training_texts

DOCS_DIR = pathlib.Path("docs/")
MANIFEST = "labels.csv"
# CSV columns: filename, category, subcategory,
keywords
# keywords column uses "|" as delimiter (e.g.
medical|claim)

clear_all_training_data()        # optional: start
fresh

with open(MANIFEST, newline="") as f:
    for row in csv.DictReader(f):
        file_bytes = (DOCS_DIR /
row["filename"]).read_bytes()
        text = extract_text(file_bytes,
row["filename"])
        kws = [k.strip() for k in
               row.get("keywords", "").split("|")
if k.strip()]
        add_training_sample(
            text=text,
            filename=row["filename"],
            category=row["category"],
            subcategory=row["subcategory"],
            keywords=kws,
        )
        print(f"  ingested: {row['filename']}")

# ── One single training run across all N samples
──
samples = load_training_data()
texts   = prepare_training_texts(samples)
labels  = [s["compound_label"] for s in samples]

clf = InsuranceClassifier("Logistic Regression")
clf.train(texts, labels)
clf.save("models/classifier.joblib")
print(f"Done. Accuracy: {clf.training_accuracy:.
1%}"
      f" | Samples: {clf.n_samples_trained}")
```

CSV Manifest Format

```
filename,category,subcategory,keywords
claim_q1.pdf,Claims,Medical Records,medical|
diagnosis
policy_auto.docx,Policy,Auto Policy,premium|vehicle
invoice_mar.xlsx,Financial,Premium Invoice,invoice|
amount due
```

DIAGRAM 05

Bulk Training Flow



Documents enter the pipeline from three possible sources: a labeled folder tree, a CSV manifest, or an existing JSON store. The batch ingestion script loops over all source files, extracts text, and appends each sample to `training_data.json`. Once all N samples are ingested, a single `InsuranceClassifier.train()` call fits the full pipeline and serializes the trained model — no repeated fitting, no manual UI clicks.